

SmartRecruitz — Product & Features Document

Table of Contents

1. Executive Summary
 2. Goals & Non-Goals
 3. Personas
 4. The Candidate Journey — the spine every feature ties to
 5. Feature: Conversational Onboarding (Chatbot)
 6. Feature: Agent 1 — Resume Parsing & Scoring
 7. Feature: Agent 2 — Duplicate Detection
 8. Feature: Agent 3 — Identity Verification
 9. Feature: Agent 4 — Interview Question Generation
 10. Feature: Agent 5 — Interview Scoring & Readiness
 11. Feature: LinkedIn Consistency Cross-Check
 12. Feature: Hiring Manager Portal — Search, Filters & Decisions
 13. Feature: Location Capture & Location Search
 14. Architecture & Deployment
 15. Cross-feature loops
-

1. Executive Summary

SmartRecruitz is a **verified-talent infrastructure platform**. It turns an unstructured candidate — a resume, an ID, a short interview — into a **structured, verified, ranked profile** that a hiring manager can trust and search in seconds.

The product does four things:

- A **guided chatbot onboarding** that walks a candidate from "hello" to a scored profile in ~10 minutes
- **Five specialised AI agents** that parse, de-duplicate, verify, interview, and score every candidate
- A **hiring-manager portal** that searches the verified talent pool by JD, skills, and location
- A **readiness score + recommendation** (L1 pass/fail) so recruiters review evidence, not claims

This document goes feature-by-feature through what each surface does, why it exists, who uses it, and how we know it's working. It deliberately avoids implementation detail — that lives in code.

2. Goals & Non-Goals

Goals (the bar we hold the product to)

#	GOAL	HOW WE KNOW IT'S WORKING
G1	Turn an applicant into a verified, structured profile with no manual data entry	≥ 90% of candidates reach a scored profile from resume + ID alone
G2	Cut recruiter screening time per candidate by ≥ 70%	Time from "applicant" to "shortlist decision" measured per role
G3	Every profile carries an objective readiness signal , not self-reported claims	100% of talent-pool candidates have an L1 score + recommendation
G4	Keep the pool clean — no duplicates, no unverified identities	< 2% duplicate rate; 100% of pool candidates ID-verified
G5	Recruiters find the right candidate by JD + skills + location in one search	Median time-to-first-shortlist < 5 minutes

Non-Goals (what we explicitly don't do)

- We **don't** run payroll, offers, or onboarding-after-hire. We stop at "verified, ranked, contactable."
- We **don't** auto-reject candidates. Every score is advisory; the human makes the call.
- We **don't** proctor or surveil. We verify identity once; we don't watch webcams or keystrokes.
- We **don't** scrape the open web for candidates. Candidates opt in through onboarding.

3. Personas

P1 · Candidate

- A job-seeker completing a one-time verification to enter a talent pool.
- **Wants:** a fast, guided flow; fair scoring; to not re-type what's already on their resume.
- **Doesn't want:** long forms, opaque rejection, exposure of weaknesses to other candidates.

P2 · Hiring Manager / Recruiter

- Searches the verified pool against an open role. Reviews 5–50 candidates per requisition.
- **Wants:** to find skill + location matches fast, with evidence (interview score, resume analysis) attached.
- **Doesn't want:** unverified profiles, duplicates, or a wall of identical-looking resumes.

P3 · Talent / Program Admin

- Owns pool quality and throughput — how many candidates verified, average readiness, domain mix.
- **Wants:** a clean dashboard, the ability to approve/reject, and re-interview controls.

- **Doesn't want:** a noisy pool or false "verified" badges that erode recruiter trust.

4. The Candidate Journey

Every feature ties to one linear journey. Understanding it makes the rest of the document land.

```
Register → Resume parse → Duplicate check → LinkedIn cross-check  
→ ID verification → Interview (Q-gen + scoring) → Talent pool  
→ Hiring-manager search → Approve / Reject
```

A candidate starts in **staging** (raw, unverified). They are promoted to the **main talent pool** only after parsing and a unique-identity check pass. Each AI agent owns one hop in this chain and writes structured data the next hop (and the recruiter) reads.

Status levels a candidate moves through: **PENDING** → **VERIFIED** → interview **SCORED** → L1 **PASSED** / **FAILED** → recommendation **PROCEED_TO_L2** / **MANUAL_REVIEW** / **REJECT** .

5. Feature: Conversational Onboarding (Chatbot)

Why this exists

Traditional application forms are abandoned. A chat that asks one thing at a time, confirms what it parsed, and explains each step converts far better — and produces cleaner data because the candidate verifies it inline.

What it does

A multi-step chatbot guides the candidate through the entire journey in one session: identity details → resume upload → review → URLs & cross-check → ID documents → video/text interview → result. Progress is persisted, so a candidate can close the tab and resume exactly where they left off.

Who uses it

Candidates — primary. It is the only surface a candidate touches; everything else happens behind it.

Deep behaviour

- It collects, in order: name, email, phone, target role, CTC, notice period, working status, **current location**, and preferred locations.
- It **confirms parsed data** ("here's what I found in your resume") before proceeding — the candidate can re-upload.
- Session state is persisted; on return, it **resumes at the exact step** rather than restarting.
- Long-running agent calls (parse, verify, score) are shown as live "analyzing..." states with polling, never a frozen screen.

What success looks like

- $\geq 80\%$ of candidates who start onboarding reach a scored profile
- Median completion time 8–12 minutes
- $< 5\%$ of sessions abandoned at the resume-confirm step

Edge cases worth knowing

- **Returning candidate** — email lookup restores their ref and jumps to the right step.
 - **Agent call fails mid-flow** — the candidate's input is saved; they see a graceful retry, not a lost session.
-

6. Feature: Agent 1 — Resume Parsing & Scoring

Why this exists

The resume is the densest signal a candidate gives us. Parsing it into structured skills, experience, and education — and scoring its quality — removes manual data entry and gives every downstream agent something clean to work with.

What it does

Takes the uploaded resume (PDF / DOCX / image) and extracts a normalised profile: full name, title, total experience, primary domain, skills (normalised against a taxonomy), experience history, education, and GitHub/LinkedIn URLs — plus an overall resume score with sub-scores and analytics.

Deep behaviour

- Image/scanned PDFs are handled via vision; text PDFs via text extraction — whichever yields cleaner structure.
- Skills are **normalised** ("ReactJS", "React.js" → "React") so search and matching are consistent.
- Produces analytics the recruiter sees later: tenure stats, employment gaps, product/service/GCC/startup experience split.

What success looks like

- $\geq 90\%$ of resumes parse into a confirmable profile on first upload
- Candidate confirms the parsed card without re-upload $\geq 80\%$ of the time

Edge cases worth knowing

- **Unparseable / image-only resume** — vision fallback; if still poor, candidate is asked to re-upload a cleaner file.
 - **Missing URLs** — the chatbot asks for GitHub/LinkedIn explicitly rather than leaving them blank.
-

7. Feature: Agent 2 — Duplicate Detection

Why this exists

A talent pool full of the same person registered three times is worthless to a recruiter. We catch re-registrations before a candidate is promoted to the main pool.

What it does

Before promotion, the new profile is compared against existing candidates — first a cheap database pass (fuzzy name match + email/phone), then an LLM comparison on the small set of near-matches. Returns **UNIQUE**, **DUPLICATE**, or **UNCERTAIN** with reasoning.

Deep behaviour

- The DB filter narrows thousands of candidates to a handful before any LLM call — cheap and fast.
- Only **UNIQUE** candidates are promoted to the main talent pool; duplicates are flagged, not silently dropped.

What success looks like

- < 2% duplicate rate in the live pool; < 1% false-duplicate blocks
-

8. Feature: Agent 3 — Identity Verification

Why this exists

"Verified" has to mean something. An ID check once, at onboarding, lets a recruiter trust that the person behind the profile is real.

What it does

The candidate uploads a government ID (Aadhaar / PAN / Passport). Vision extracts the name, DOB, and document data, checks it against the claimed name, and assesses document authenticity — returning **VERIFIED**, **FAILED**, or **MANUAL_REVIEW** with a confidence score and flags.

Deep behaviour

- Mandatory documents (Aadhaar, PAN) are ordered; Passport is optional.
- Name mismatch lowers confidence and routes to **MANUAL_REVIEW** rather than auto-failing.

What success looks like

- 100% of pool candidates have at least the mandatory documents verified; < 5% land in manual review
-

9. Feature: Agent 4 — Interview Question Generation

Why this exists

Generic question banks don't probe a specific candidate. Questions generated from *this* candidate's skills and experience are sharper and harder to game.

What it does

Reads the candidate's parsed profile and generates a tailored set of screening questions across categories (experience verification, technical, behavioural), each with difficulty, target skill, expected-answer points, and time estimate.

Deep behaviour

- Questions target the candidate's actual claimed skills, so weak claims surface in the answers.
 - Each question carries an expected-answer framework that Agent 5 scores against.
-

10. Feature: Agent 5 — Interview Scoring & Readiness

Why this exists

A recruiter needs one honest number: is this candidate ready for a real interview? Agent 5 turns a transcript into a readiness score and a clear recommendation.

What it does

Scores the interview transcript against the expected-answer framework, producing an overall 0–100 score, an L1 status (**PASSED** / **FAILED**), a recommendation (**PROCEED_TO_L2** / **MANUAL_REVIEW** / **REJECT**), per-dimension breakdown, and per-question validation.

Deep behaviour

- **Honest scoring**: weak answers score 30–50, excellent 80–95; >95 is reserved for exceptional depth.
- The result is cached on the candidate so the pool and recruiter read it instantly — no re-scoring.
- Admins can grant a **re-interview**; the candidate is offered a fresh attempt.

What success looks like

- ≥ 70% human–AI agreement on the pass/fail call when a recruiter also reviews
-

11. Feature: LinkedIn Consistency Cross-Check

Why this exists

Resumes are self-reported. Cross-checking against a public LinkedIn profile surfaces inflated titles or tenure before a recruiter spends time.

What it does

Fetches the candidate's LinkedIn profile and compares it to the parsed resume — returning a match score, a consistency verdict, and a list of specific discrepancies (company, duration, title mismatches). The flow continues regardless; the recruiter sees the mismatches and decides.

What success looks like

- Discrepancies are surfaced, not hidden; recruiters report them as "useful" in review
-

12. Feature: Hiring Manager Portal — Search, Filters & Decisions

Why this exists

All the verification in the world is useless if a recruiter can't find the right person. The portal turns the verified pool into a searchable, rankable shortlist.

What it does

Three surfaces: a **dashboard** (pool stats, domain breakdown, recent candidates), a **talent-pool browser** (filter by domain, score, experience, company type, skills, location), and a **JD search** (paste a JD or skills → ranked candidates by match). A **candidate detail** page shows the full profile, resume analysis, interview results, and cross-check. The admin can **approve** / **reject** from here.

Deep behaviour

- JD text is auto-parsed into mandatory vs preferred skills; mandatory is a hard filter, preferred boosts ranking.
- Match score is a transparent weighted blend (mandatory 70% / preferred 30%), renormalised to 0–100.
- Each result card carries evidence: readiness score, L1 status, resume score, skills, company-type, location.

What success looks like

- Median time-to-first-shortlist < 5 minutes
 - ≥ 70% human–AI agreement on the top-ranked candidate's relevance
-

13. Feature: Location Capture & Location Search

Why this exists

Location is one of the first filters a recruiter applies and one of the most common reasons a strong candidate is the wrong fit. We capture it once and make it searchable.

What it does

During onboarding the chatbot asks the candidate's **current location** (and, separately, the locations they're open to working in). In the hiring-manager JD search, a **Location** filter matches a typed place against the candidate's **current OR preferred** location (case-insensitive), as a hard filter layered on top of skill ranking.

Deep behaviour

- Two distinct fields are stored: `location` (where they are) and `preferred_location` (where they'll work).
- A search for "Bangalore" matches a candidate based in Gurugram who is open to Bangalore — via preferred location.
- Location can be used on its own (location-only search) or combined with skills/JD.

What success looks like

- Location filter used in $\geq 50\%$ of searches; zero false matches on exact-city queries

14. Architecture & Deployment

- **Backend:** FastAPI (Python), async, with five AI agents orchestrated through a service → repository layering. Background work is tracked as polled tasks.
- **Frontend:** Next.js + React + Tailwind, two surfaces — the candidate onboarding chatbot and the hiring-manager portal.
- **AI:** OpenAI models for parsing, vision verification, question generation, and scoring.
- **Data:** PostgreSQL. SmartRecruitz tables share the platform database under the `application` schema with an `lms_recruit_` prefix, alongside the LMS tables.
- **Deployment:** runs as a **standalone app on its own domain** with its own login; the parent LMS links across to it.

15. Cross-feature loops

The product is more than the sum of its agents because each one feeds the next.

```
Resume parse → fills skills/experience
  → Question generation targets those exact skills
  → Interview scoring tests the claims
  → Readiness score + recommendation attach to the profile
  → Hiring-manager search ranks on verified skills + location
  → Recruiter shortlists on evidence, not claims
```


ID verify + Duplicate check → only real, unique candidates enter the pool
→ every search result is trustworthy by construction

A competitor can bolt "AI resume screening" onto an ATS, but the value here is the **chained verification**: by the time a candidate appears in a recruiter's search, they've been parsed, de-duplicated, identity-verified, interviewed, and scored — and the recruiter sees the whole evidence trail.

End of document.